

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

MÔN AN TOÀN VÀ BẢO MẬT HỆ THỐNG THÔNG TIN



BÁO CÁO BÀI TẬP NHÓM

Tấn công Từ chối Dịch vụ Phân tán (DDoS) với Botnet mô phỏng

Giảng viên hướng dẫn : Nguyễn Huy Trung

Họ và tên sinh viên : Nguyễn Nam Khánh
: Trương Duy Mạnh

Lớp tín chỉ : INT1303
Nhóm : 02

Hà Nội – 2026

MỤC LỤC

1. Tổng quan & kiến trúc Botnet

1.1 Botnet là gì?

1.2 Thành phần chính

1.3 Kiến trúc botnet

2. Tấn công DDoS và mức độ nguy hiểm

2.1 DDoS là gì? 2

2.2 Các loại phổ biến

2.3 Mức độ nguy hiểm

3. Giải pháp phòng chống

3.1 Phòng ngừa

3.2 Giảm thiểu

3.3 Phát hiện

4. Mô phỏng Botnet

4.1 Kiến trúc lab

4.2 Target server (giả lập)

4.3 Bot (client gửi request)

4.4 C&C Server (điều khiển bot)

4.5 Dockercompose.yml

4.6 Kết quả demo

5. Đánh giá & kết luận

5.1 Kết quả

5.2 Hạn chế mô phỏng

5.3 Kết luận

1. Tổng quan & kiến trúc Botnet

1.1 Botnet là gì?

Botnet (viết tắt của “Bot Network”) là một mạng lưới các thiết bị được kết nối Internet nhưng đã bị nhiễm phần mềm độc hại và bị kiểm soát từ xa bởi tin tặc mà người dùng không hề hay biết. Các thiết bị này có thể bao gồm máy tính cá nhân, máy chủ, điện thoại thông minh hoặc các thiết bị IoT như camera, router.

Mỗi thiết bị bị nhiễm được gọi là một **bot** hoặc **zombie**. Khi nhiều bot được liên kết với nhau, chúng tạo thành một hệ thống có quy mô lớn, có thể lên đến hàng nghìn hoặc hàng triệu thiết bị trên toàn cầu. Nhờ số lượng lớn này, botnet có thể thực hiện các cuộc tấn công với cường độ cao và phạm vi rộng.

Một số khái niệm quan trọng trong botnet:

- **Bot (Zombie):** Thiết bị bị nhiễm và bị điều khiển từ xa
- **Botnet:** Tập hợp nhiều **bot** hoạt động đồng bộ
- **Bot Herder:** Kẻ điều khiển botnet
- **C&C Server (Command & Control):** Máy chủ trung tâm gửi lệnh đến các bot

Botnet có tính chất **phân tán**, giúp tăng sức mạnh tấn công theo cấp số nhân và gây khó khăn cho việc phát hiện cũng như ngăn chặn.

1.2 Thành phần chính

Một hệ thống botnet cơ bản bao gồm:

- **Attacker (Kẻ tấn công):** Người điều khiển hệ thống
- **C&C Server:** Trung tâm điều phối và gửi lệnh
- **Bots (Zombie):** Các thiết bị bị nhiễm và thực thi lệnh
- **Target (Mục tiêu):** Hệ thống hoặc dịch vụ bị tấn công

Các thành phần này phối hợp với nhau để tạo nên một hệ thống tấn công có tổ chức và hiệu quả.

1.3 Kiến trúc Botnet

Botnet có thể được xây dựng theo nhiều mô hình khác nhau:

Mô hình tập trung (Centralized)

- Các bot kết nối đến một C&C server duy nhất
- Dễ triển khai và quản lý
- Tuy nhiên dễ bị phát hiện và vô hiệu hóa

Mô hình ngang hàng (P2P – Peer-to-Peer)

- Các bot giao tiếp trực tiếp với nhau
- Không phụ thuộc vào máy chủ trung tâm
- Khó bị triệt phá hơn

Mô hình lai (Hybrid)

- Kết hợp giữa **centralized** và **P2P**
- Tăng tính linh hoạt và độ bền của hệ thống

Trong phạm vi báo cáo này, mô hình **Centralized** được sử dụng để mô phỏng do tính đơn giản và dễ triển khai.

1.4 Cơ chế hoạt động của Botnet

Để hiểu rõ kiến trúc botnet, cần xem xét cách các thành phần tương tác với nhau trong thực tế. Quá trình hoạt động của botnet thường gồm ba giai đoạn chính:

- **Giai đoạn lây nhiễm:**
Thiết bị bị nhiễm mã độc thông qua email lừa đảo, khai thác lỗ hổng hoặc phần mềm độc hại.
- **Giai đoạn kết nối:**
Sau khi nhiễm, bot sẽ tự động kết nối đến **C&C Server** để nhận lệnh điều khiển.

- **Giai đoạn thực thi:**

*Khi nhận lệnh, các bot sẽ đồng loạt thực hiện các hành vi như tấn công **DDoS**, gửi spam hoặc đánh cắp dữ liệu.*

2. Tấn công DDoS và mức độ nguy hiểm

2.1 Tấn công DDoS là gì?

Tấn công Từ chối Dịch vụ Phân tán (DDoS – Distributed Denial of Service) là một hình thức tấn công mạng nhằm làm gián đoạn hoạt động bình thường của máy chủ, dịch vụ hoặc hệ thống bằng cách gửi một lượng lớn lưu lượng truy cập đến mục tiêu, khiến hệ thống bị quá tải.

Các cuộc tấn công này thường sử dụng nhiều thiết bị bị xâm nhập để tạo ra lưu lượng tấn công, khiến việc ngăn chặn trở nên khó khăn.

2.2 Cách thức hoạt động của tấn công DDoS

Các cuộc tấn công DDoS thường được thực hiện thông qua mạng botnet.

Quy trình hoạt động:

- *Kẻ tấn công xây dựng mạng botnet*
- *Gửi lệnh điều khiển từ xa*
- *Các bot đồng loạt gửi request đến mục tiêu*
- *Hệ thống bị quá tải và ngừng hoạt động*

Do mỗi bot là một thiết bị hợp pháp, việc phân biệt lưu lượng tấn công và lưu lượng bình thường trở nên rất khó khăn.

2.3 Phân loại tấn công DDoS

Các cuộc tấn công DDoS thường được phân loại dựa trên mô hình OSI model:

Tấn công lớp ứng dụng (Layer 7)

- *Nhắm vào ứng dụng web*
- *Gửi nhiều request HTTP*
- *Khó phát hiện*

Tấn công giao thức (Layer 3/4)

- *Khai thác giao thức TCP, UDP*
- *Làm cạn tài nguyên hệ thống*

Tấn công theo khối lượng

- *Làm nghẽn băng thông*
- *Gửi lượng lớn dữ liệu đến mục tiêu*

2.4 Cách nhận biết tấn công DDoS

Một số dấu hiệu nhận biết:

- *Website chậm hoặc không truy cập được*
- *Lưu lượng truy cập tăng đột biến*
- *Nhiều request từ cùng IP hoặc hành vi giống nhau*
- *Mẫu truy cập bất thường theo thời gian*

2.5 Mức độ nguy hiểm

Tấn công DDoS có thể gây ra nhiều hậu quả:

- *Gián đoạn dịch vụ*
- *Thiệt hại tài chính*
- *Ảnh hưởng uy tín doanh nghiệp*
- *Có thể che giấu các cuộc tấn công khác*

Một ví dụ tiêu biểu là Dyn DNS DDoS attack, đã làm gián đoạn nhiều dịch vụ lớn trên toàn cầu.

3. Giải pháp phòng chống

3.1 Phòng ngừa

Các biện pháp phòng ngừa bao gồm:

- *Cập nhật và vá lỗ hổng hệ thống*
- *Sử dụng firewall*
- *Áp dụng CAPTCHA*
- *Giới hạn request (rate limiting)*

3.2 Giảm thiểu

Khi tấn công xảy ra:

- *Sử dụng CDN như Cloudflare*
- *Load balancing (cân bằng tải)*
- *Auto scaling hệ thống*
- *Chặn IP bất thường*

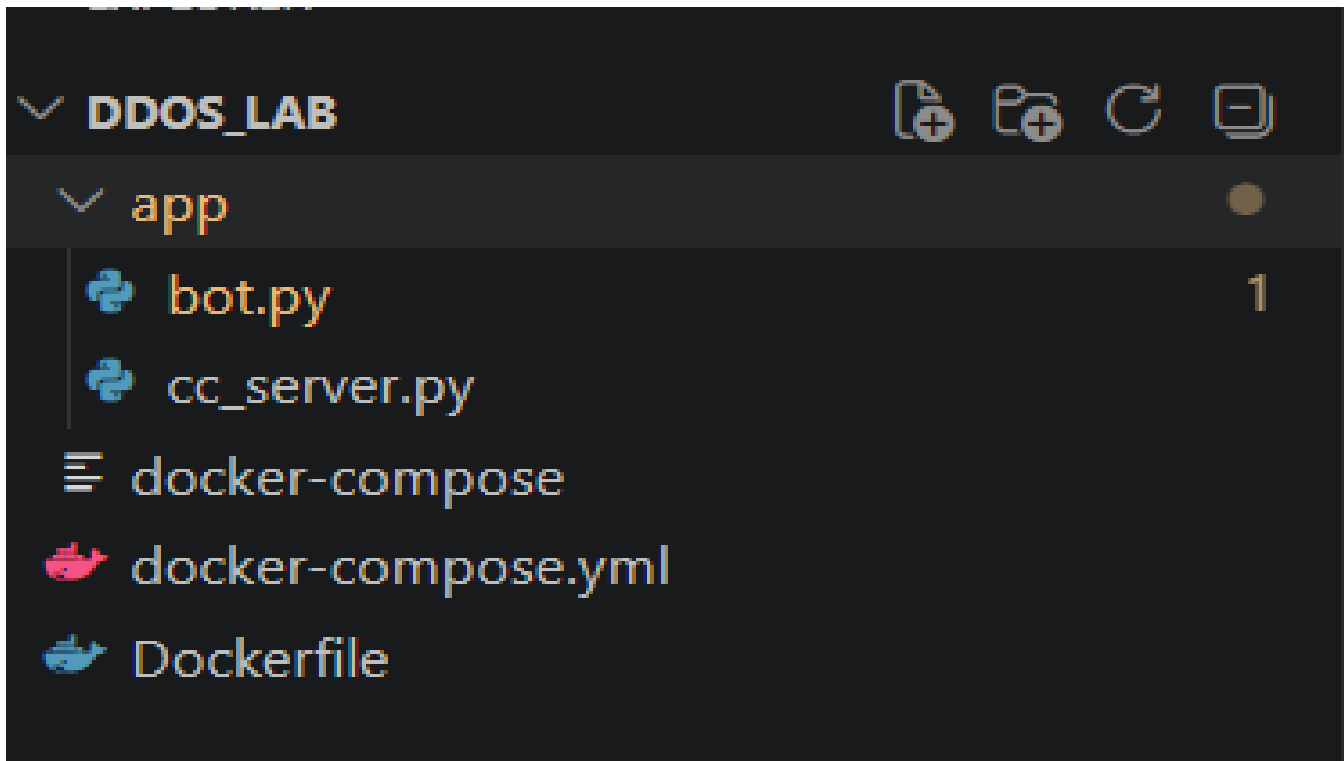
3.3 Phát hiện

Phát hiện sớm giúp giảm thiểu thiệt hại:

- *IDS/IPS*
- *Giám sát lưu lượng mạng*
- *Phân tích hành vi bất thường*
- *Ứng dụng AI trong bảo mật*

4. Mô phỏng Botnet (DDOS_LAB)

4.1 Kiến trúc lab



Hình 1: Cấu trúc thư mục

Hệ thống được thiết kế theo mô hình Client-Server tập trung (Centralized Architecture), mô phỏng một mạng lưới Botnet thực tế hoạt động trong môi trường ảo hóa cô lập bằng Docker.

1. Sơ đồ thực thể (Entities)

Kiến trúc Lab bao gồm 3 thực thể chính hoạt động trong một mạng nội bộ ảo (Docker Bridge Network):

- C&C Server (Command & Control): Đóng vai trò là trung tâm điều khiển. Nó lắng nghe kết nối từ các Bot và phát lệnh tấn công đồng loạt.
- Botnet (Zombie Nodes): Tập hợp các mã độc (scripts) chạy trên nhiều container khác nhau. Các Bot này chủ động kết nối về C&C Server và chờ đợi chỉ thị.
- Victim (Target): Một máy chủ web (Nginx) đóng vai trò là nạn nhân, chịu trách nhiệm xử lý các yêu cầu HTTP từ người dùng hợp lệ và cả các cuộc tấn công.

2. Cấu trúc thư mục và Chức năng thành phần

kiến trúc được tổ chức như sau:

Thành phần	Đường dẫn	Chức năng kỹ thuật
Thư mục ứng dụng	/app	Chứa toàn bộ logic mã nguồn Python của hệ thống.
Mã nguồn Bot	bot.py	Sử dụng socket để nhận lệnh và threading kết hợp requests để thực thi tấn công HTTP Flood.
Mã nguồn Điều khiển	cc_server.py	Quản lý danh sách kết nối (Socket list) và giao diện dòng lệnh cho người điều hành.
Cấu hình Container	Dockerfile	Định nghĩa môi trường chạy Python, cài đặt thư viện cần thiết (requests) và đóng gói mã nguồn vào Image.
Điều phối hệ thống	docker-compose.yml	Thiết lập hạ tầng mạng, định nghĩa số lượng Bot

		(replicas) và liên kết các dịch vụ với nhau.
--	--	--

3. Luồng vận hành của kiến trúc (Workflow)

Kiến trúc này hoạt động theo 4 bước tuần tự:

1. Khởi tạo hạ tầng (Orchestration): Docker Compose tạo ra một mạng ảo **ddos_network**. Container **victim-web** và **cc-server** được dựng lên trước, sau đó các container **bot** được nhân bản (ví dụ: 5-10 bots).
2. Thiết lập kết nối (Beaconing): Các Bot sử dụng giao thức TCP/IP kết nối đến **cc-server:9999**. Server lưu trữ thông tin socket của từng Bot vào bộ nhớ.
3. Ra lệnh (Command Issuance): Quản trị viên nhập URL mục tiêu vào console của **cc-server**. Server duyệt qua danh sách các Bot đang kết nối và gửi chuỗi dữ liệu (Target URL) xuống.
4. Thực thi tấn công (Flooding): Các Bot nhận lệnh, ngay lập tức kích hoạt cơ chế đa luồng để gửi liên tiếp hàng nghìn gói tin HTTP GET tới **victim-web**, gây ra tình trạng nghẽn tài nguyên trên máy chủ nạn nhân.

4.2 Target server (giả lập)

Thành phần đóng vai trò nạn nhân trong hệ thống Lab là một máy chủ Web (Web Server) được container hóa. Đây là đối tượng chịu tác động trực tiếp từ các luồng traffic khổng lồ do mạng Botnet tạo ra.

4.2.1. Đặc điểm kỹ thuật

- **Phần mềm:** Nginx (Engine X) phiên bản **alpine**.
- **Hệ điều hành nền:** Alpine Linux (Một bản phân phối siêu nhẹ, giúp tối ưu hóa hiệu suất và tài nguyên trong môi trường Docker).

- **Cổng dịch vụ:** * Nội bộ: Port **80** (Giao thức HTTP).
 - Công khai: Port **9090** (Ánh xạ ra máy host để kiểm tra trạng thái từ trình duyệt).

4.2.2. Cơ chế xử lý yêu cầu (Request Handling)

Nginx hoạt động theo cơ chế **Asynchronous Event-driven** (Hướng sự kiện bất đồng bộ).

- Khi một Bot gửi một **GET /**, Nginx sẽ tiếp nhận yêu cầu, xử lý tệp tin **index.html** tĩnh và gửi phản hồi (Response) về cho Bot.
- **Điểm yếu giả lập:** Trong cấu hình mặc định, Nginx có giới hạn về số lượng kết nối đồng thời (**worker_connections**). Khi số lượng Bot vượt quá ngưỡng xử lý này, các tiến trình (Worker processes) sẽ bị quá tải.

4.2.3. Phân tích tác động của cuộc tấn công (HTTP Flood)

Khi cuộc tấn công diễn ra, Target Server sẽ trải qua các giai đoạn suy giảm hiệu năng sau:

1. **Giai đoạn chiếm dụng băng thông (Bandwidth Consumption):** Hàng nghìn gói tin HTTP nhỏ làm đầy băng thông mạng nội bộ của Docker Bridge.
2. **Giai đoạn cạn kiệt tài nguyên CPU/RAM:** Server phải tốn tài nguyên để phân tích các Header của HTTP Request, duy trì các kết nối TCP (Keep-Alive) và phản hồi mã trạng thái (thường là 200 OK).
3. **Giai đoạn từ chối dịch vụ (DoS):** * Hàng đợi kết nối (Connection Queue) bị đầy.
 - Người dùng hợp lệ (truy cập qua port 9090) sẽ nhận được lỗi **504 Gateway Timeout** hoặc **502 Bad Gateway** do Server không còn khả năng tiếp nhận thêm yêu cầu mới.

4.2.4. Lí do chọn Nginx làm mục tiêu giả lập

- **Tính thực tế:** Nginx là Web Server phổ biến nhất thế giới hiện nay, thường xuyên là mục tiêu của các cuộc tấn công DDoS thực tế.
- **Khả năng quan sát:** Nginx cung cấp các file Log (**access.log** và **error.log**). Trong Lab này, bạn có thể quan sát trực tiếp luồng tấn công bằng lệnh: **docker logs -f victim-web** (Bạn sẽ thấy hàng nghìn dòng log HTTP GET chạy liên tục khi Bot kích hoạt).

4.3 Bot (client gửi request)

Bot là thành phần quan trọng nhất trong việc tạo ra lưu lượng tấn công khổng lồ. Trong mô hình Lab này, Bot được thiết kế là một mã nguồn Python tối giản nhưng hiệu quả, hoạt động theo cơ chế kích hoạt từ xa.

4.3.1. Quy trình vận hành (Lifecycle)

Mỗi Bot trong hệ thống trải qua ba giai đoạn hoạt động chính:

1. **Giai đoạn Đăng ký (Check-in):** Ngay khi khởi tạo, Bot sử dụng thư viện **socket** để thiết lập kết nối TCP tới **cc-server** qua cổng **9999**. Việc này giúp C&C Server nắm bắt được số lượng "quân đội" hiện có.
2. **Giai đoạn Lắng nghe (Command Listening):** Bot rơi vào trạng thái chờ (Idle). Nó sử dụng hàm **client.recv()** để chặn (block) và đợi gói tin chứa URL mục tiêu từ Server.
3. **Giai đoạn Công kích (Flooding):** Khi nhận được URL, Bot ngay lập tức chuyển trạng thái sang thực thi tấn công liên tục.

4.3.2. Kỹ thuật Tấn công Đa luồng (Multi-threading)

Đây là "vũ khí" chính của Bot để tối ưu hóa sức mạnh xử lý:

- **Cơ chế:** Thay vì gửi từng request một và đợi phản hồi từ Server (Sync), Bot sử dụng module **threading** của Python để tạo ra hàng chục luồng chạy song song.
- **Hiệu quả:** Mỗi luồng sẽ gửi một yêu cầu **requests.get(url)** độc lập. Điều này cho phép một Bot đơn lẻ có thể tận dụng tối đa băng thông mạng và tài nguyên CPU của container để gửi hàng trăm request mỗi giây.
- **Tham số **daemon=True**:** Đảm bảo các luồng tấn công sẽ tự động bị tiêu hủy khi chương trình chính dừng lại, giúp quản lý tài nguyên hệ thống hiệu quả.

4.3.3. Phân tích Giao thức Tấn công (HTTP Flood)

Bot thực hiện kiểu tấn công **HTTP Get Flood** nhằm vào tầng ứng dụng:

- **Thư viện sử dụng:** **requests** (một thư viện HTTP chuẩn cho Python).
- **Đặc điểm gói tin:** Mỗi request gửi đi là một yêu cầu hợp lệ về mặt cấu trúc (đầy đủ Header, User-Agent mặc định).
- **Xử lý lỗi:** Trong mã nguồn có khối **try...except**. Điều này cực kỳ quan trọng vì khi Server nạn nhân bắt đầu quá tải và từ chối kết nối, Bot sẽ không bị "văng" (crash) mà chỉ bỏ qua lỗi và tiếp tục gửi request mới ngay lập tức.

4.3.4. Vai trò của Docker trong việc mô phỏng Bot

Việc sử dụng Docker giúp nâng tầm phân tích kỹ thuật của Bot:

- **Tính độc lập:** Mỗi Bot chạy trên một Container riêng biệt với địa chỉ IP nội bộ riêng. Điều này mô phỏng chính xác một cuộc tấn công Phân tán

(Distributed), nơi Server nạn nhân thấy traffic đến từ nhiều nguồn khác nhau chứ không phải từ một máy duy nhất.

- Khả năng nhân bản (Scaling): Nhờ tham số **replicas: 5** trong **docker-compose.yml**, ta có thể dễ dàng nhân bản sức mạnh tấn công lên gấp nhiều lần mà không cần viết lại mã nguồn.

* **Thực tế:** Bot thường có thêm cơ chế "ẩn mình" (Obfuscation), thay đổi **User-Agent** ngẫu nhiên để tránh bị nhận diện bởi các hệ thống IDS/IPS, và có khả năng tự động kết nối lại (Reconnect) nếu C&C Server thay đổi IP.

4.4 C&C Server (điều khiển bot)

C&C Server trong Lab này được thiết kế theo mô hình Tập trung (Centralized). Nó đóng vai trò là cầu nối giữa người điều khiển và hạ tầng Botnet, đảm bảo các lệnh tấn công được truyền đi chính xác và đồng bộ.

4.4.1. Cơ chế Quản lý Kết nối (Socket Management)

C&C Server vận hành dựa trên giao thức TCP tại tầng Giao vận (Transport Layer) để đảm bảo tính tin cậy của dòng lệnh:

- **Lắng nghe (Listening):** Server mở một Socket trên cổng **9999** và liên tục lắng nghe các yêu cầu kết nối từ mạng nội bộ.
- **Danh sách Bot (Bot Registry):** Khi một Bot kết nối thành công, đối tượng kết nối (**conn**) và địa chỉ IP (**addr**) của Bot đó sẽ được lưu trữ vào một mảng (List) có tên là **bots**.
- **Tính thời gian thực:** Việc lưu trữ các Socket đang hoạt động (Active Sockets) cho phép Server duy trì liên lạc liên tục với Bot mà không cần phải thiết lập lại kết nối mỗi khi ra lệnh.

4.4.2. Giao diện Điều khiển (Interactive Interface)

Khác với các Bot chạy ngầm, C&C Server cung cấp một giao diện dòng lệnh (CLI) để người quản trị tương tác:

- **Điều kiện kích hoạt:** Trong mã nguồn, Server được thiết lập chỉ cho phép nhập lệnh khi số lượng Bot đạt một ngưỡng nhất định (ví dụ: `>= 3 bots`). Điều này mô phỏng chiến thuật thực tế: kẻ tấn công thường đợi mạng Botnet đủ lớn mới bắt đầu thực hiện đợt tấn công tổng lực.
- **Truyền tin quảng bá (Broadcasting):** Khi người dùng nhập một URL mục tiêu, Server sẽ sử dụng vòng lặp để duyệt qua danh sách `bots` và gửi dữ liệu tới từng Bot một thông qua phương thức `send()`. Đây chính là cơ chế tạo ra sự đồng bộ trong tấn công DDoS.

4.4.3. Phân tích Luồng Dữ liệu (Control Flow)

Luồng xử lý tại C&C Server diễn ra như sau:

1. **Giai đoạn Chờ:** Sử dụng hàm `accept()` để chấp nhận các "Zombie" mới gia nhập mạng lưới.
2. **Giai đoạn Ra lệnh:** Nhận đầu vào từ bàn phím (URL mục tiêu).
3. **Giai đoạn Phát tán:** Chuyển đổi chuỗi URL thành định dạng Byte (`encode()`) và đẩy vào đường truyền TCP.
4. **Giai đoạn Giám sát:** Hiển thị thông báo xác nhận: "Đã phát lệnh tấn công tới X bots!".

4.4.4. Vai trò của Docker trong việc duy trì C&C

- **Tính sẵn sàng:** Trong `docker-compose.yml`, C&C Server được cấu hình với `stdin_open: true` và `tty: true`. Điều này cho phép

người dùng có thể "attach" vào container bất cứ lúc nào để ra lệnh, giống như việc truy cập vào một máy chủ từ xa thông qua SSH.

- **Định danh dịch vụ:** Nhờ cơ chế DNS nội bộ của Docker, C&C Server luôn có định danh là **cc-server**, giúp các Bot dễ dàng tìm thấy đầu não mà không cần quan tâm đến địa chỉ IP động của container.

Hạn chế (Single Point of Failure): Vì đây là mô hình tập trung, nếu C&C Server bị tắt hoặc bị chặn IP, toàn bộ mạng Botnet sẽ mất khả năng điều khiển.

Thực tế: Các mạng Botnet hiện đại thường dùng mô hình **P2P (Peer-to-Peer)** hoặc **Domain Generation Algorithms (DGA)** để ẩn giấu C&C Server, khiến các cơ quan an ninh mạng rất khó để đánh sập "đầu não".

4.5 Dockercompose.yml (Hạ Tầng Mạng Ảo)

Tập **docker-compose.yml** đóng vai trò là "bản thiết kế" kỹ thuật, định nghĩa cách các dịch vụ (Services), mạng lưới (Networks) và các biến môi trường tương tác với nhau.

4.5.1. Quản lý Mạng nội bộ (Docker Network)

Hệ thống sử dụng một mạng riêng biệt có tên là **ddos_network** với driver là **bridge**.

- **Tính cô lập:** Mạng này tách biệt hoàn toàn lưu lượng tấn công khỏi mạng vật lý của máy tính chủ, đảm bảo an toàn trong quá trình thực nghiệm.
- **DNS nội bộ:** Docker tự động cung cấp cơ chế phân giải tên miền. Nhờ đó, các Bot có thể tìm thấy C&C Server thông qua hostname **cc-server** thay vì phải cấu hình địa chỉ IP tĩnh phức tạp.


```
version: '3.8'

networks:
  ddos_network:
    driver: bridge
```

4.5.2. Định nghĩa các Dịch vụ (Services Analysis)

A. Victim-Web (Máy chủ mục tiêu)

- **Image:** `nginx:alpine` – Lựa chọn phiên bản nhẹ nhất để tối ưu hiệu suất xử lý request.
- **Port Mapping:** `9090:80` – Ánh xạ cổng 80 của container ra cổng 9090 của máy thật. Điều này cho phép người thực hiện lab kiểm tra trạng thái "sống/chết" của website nạn nhân từ trình duyệt máy chủ (Host OS).

```
services:
  victim-web:
    image: nginx:alpine
    deploy:
      resources:
        limits:
          cpus: '0.01'
          memory: 50M
    container_name: victim-web
    networks:
      - ddos_network
    ports:
      - "9090:80"
```

B. CC-Server (Trung tâm điều khiển)

- **Build:** `.` – Chỉ thị Docker build image từ `Dockerfile` có sẵn trong thư mục hiện hành.

- **Interactive Mode:** `stdin_open: true` và `tty: true`. Đây là cấu hình quan trọng nhất, cho phép người dùng mở một terminal tương tác để nhập lệnh tấn công trực tiếp vào script Python đang chạy ngầm.
- **Depends_on:** Đảm bảo `victim-web` phải khởi động trước khi server sẵn sàng.

```
cc-server:
  build: .
  container_name: cc-server
  command: python -u cc_server.py
  stdin_open: true
  tty: true
  networks:
    - ddos_network
  depends_on:
    - victim-web
```

C. Bot (Thành phần thực thi)

- **Replicas: 5** – Đây là tham số thể hiện sức mạnh của mạng Botnet. Chỉ với một dòng lệnh, hệ thống tự động nhân bản ra 5 thực thể bot chạy song song, mỗi thực thể có một ID và tài nguyên riêng biệt.
- **Command:** `python -u bot.py`. Flag `-u` (unbuffered) rất quan trọng trong Docker để các dòng log (như "đang tấn công...") được đẩy ra console ngay lập tức mà không bị kẹt trong bộ đệm.

```
bot:
  build: .
  command: python -u bot.py
  networks:
    - ddos_network
  deploy:
    replicas: 5
  depends_on:
    - cc-server
```

4.5.3. Cơ chế Phụ thuộc và Thứ tự Khởi động (Orchestration)

Thông qua tham số **depends_on**, **docker-compose** thiết lập một trình tự logic:

1. **Web Server** khởi động trước để có mục tiêu.
2. **C&C Server** khởi động để mở cổng lắng nghe.
3. **Dàn Bot** khởi động cuối cùng để thực hiện kết nối ngược (Reverse Connection) về đầu não.

4.5.4. Lợi ích kỹ thuật của việc dùng Docker Compose trong Lab

- **Tính nhất quán:** Đảm bảo môi trường thực nghiệm của bạn và của giảng viên khi chấm bài là hoàn toàn giống nhau (cùng phiên bản Python, cùng thư viện).
- **Khả năng dọn dẹp:** Chỉ với lệnh **docker-compose down**, toàn bộ dấu vết của cuộc tấn công, các container và mạng ảo sẽ được xóa sạch, trả lại trạng thái ban đầu cho máy tính.

4.6 Kết quả demo

Mô tả chi tiết quá trình triển khai hệ thống từ lúc khởi tạo đến khi đạt được mục đích từ chối dịch vụ (DoS) trên máy chủ mục tiêu.

Bước 1: Khởi tạo hạ tầng: Sử dụng lệnh `docker-compose up --build`. Docker sẽ đọc tệp cấu hình, xây dựng Image cho Bot/Server và thiết lập mạng ảo.

```
D:\Projects\ddos_lab>docker-compose up --build
time="2026-04-05T18:13:17+07:00" level=warning msg="D:\\Projects\\ddos_lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 911B done
#1 DONE 0.0s

#2 [bot internal] load build definition from Dockerfile
#2 transferring dockerfile: 131B 0.0s done
#2 DONE 0.1s

#3 [cc-server internal] load metadata for docker.io/library/python:3.9-slim
#3 ...

#4 [auth] library/python:pull token for registry-1.docker.io
#4 DONE 0.0s

#3 [bot internal] load metadata for docker.io/library/python:3.9-slim
#3 DONE 1.8s

#5 [bot internal] load .dockerignore
#5 transferring context: 2B done
#5 DONE 0.0s

#6 [cc-server internal] load build context
#6 transferring context: 969B 0.0s done
#6 DONE 0.1s
```

```
#14 [cc-server] resolving provenance for metadata file
#14 DONE 0.0s
[+] up 10/10
✔Image ddos_lab-bot           Built           4.4s
✔Image ddos_lab-cc-server     Built           4.4s
✔Network ddos_lab_ddos_network Created         0.1s
✔Container victim-web         Created         0.5s
✔Container cc-server          Created         0.3s
✔Container ddos_lab-bot-5     Created         0.5s
✔Container ddos_lab-bot-2     Created         0.6s
✔Container ddos_lab-bot-1     Created         0.5s
✔Container ddos_lab-bot-3     Created         0.5s
✔Container ddos_lab-bot-4     Created         0.6s
```

```

victim-web | /docker-entrypoint.sh: Configuration complete; ready for start up
victim-web | 2026/04/05 11:13:25 [notice] 1#1: using the "epoll" event method
victim-web | 2026/04/05 11:13:25 [notice] 1#1: nginx/1.29.7
victim-web | 2026/04/05 11:13:25 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
victim-web | 2026/04/05 11:13:25 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
victim-web | 2026/04/05 11:13:25 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
victim-web | 2026/04/05 11:13:25 [notice] 1#1: start worker processes
victim-web | 2026/04/05 11:13:25 [notice] 1#1: start worker process 30
victim-web | 2026/04/05 11:13:25 [notice] 1#1: start worker process 31
victim-web | 2026/04/05 11:13:25 [notice] 1#1: start worker process 32
victim-web | 2026/04/05 11:13:25 [notice] 1#1: start worker process 33
cc-server  | --- C&C Server đang đợi Bot kết nối... ---
bot-4      | Connected to C&C Server.
cc-server  | [+] Bot mới gia nhập: ('172.19.0.4', 59402)
cc-server  | Số lượng Bot hiện có: 1

bot-2      | Connected to C&C Server.
cc-server  | [+] Bot mới gia nhập: ('172.19.0.5', 35936)
cc-server  | Số lượng Bot hiện có: 2
bot-3      | Connected to C&C Server.

cc-server  | [+] Bot mới gia nhập: ('172.19.0.6', 56664)

bot-1      | Connected to C&C Server.
bot-5      | Connected to C&C Server.

```

- Kỳ vọng: Cả 7 container (1 Victim, 1 C&C, 5 Bots) đều ở trạng thái **Up**.

Bước 2: Truy cập trung tâm điều khiển: Mở một terminal mới và dùng lệnh **docker attach cc-server**.

- Kết quả: Màn hình hiện thông báo: **--- C&C Server đang đợi Bot kết nối... ---**. Khi các Bot khởi động xong, màn hình sẽ nhảy log báo danh của 5 Bot với địa chỉ IP nội bộ khác nhau.

Bước 3: Kiểm tra trạng thái tiền tấn công (Baseline): Mở trình duyệt tại máy host, truy cập **http://localhost:9090**.

- Kết quả: Trang chào mừng của Nginx hiển thị tức thì (thời gian phản hồi < 50ms).

Bước 4: Phát lệnh tấn công: Tại terminal của C&C Server, nhập URL mục tiêu: **http://victim-web** và nhấn Enter.

```

D:\Projects\ddos_lab>docker attach cc-server

[*] Đã phát lệnh tấn công tới 3 bots!
[+] Bot mới gia nhập: ('172.19.0.7', 58564)
Số lượng Bot hiện có: 4
Nhập URL mục tiêu để tấn công (hoặc 'exit'):

```

```
[*] Đã phát lệnh tấn công tới 3 bots!
[+] Bot mới gia nhập: ('172.19.0.7', 58564)
Số lượng Bot hiện có: 4
Nhập URL mục tiêu để tấn công (hoặc 'exit'): http://victim-web
[*] Đã phát lệnh tấn công tới 4 bots!
[+] Bot mới gia nhập: ('172.19.0.8', 55800)
Số lượng Bot hiện có: 5
Nhập URL mục tiêu để tấn công (hoặc 'exit'): http://victim-web
[*] Đã phát lệnh tấn công tới 5 bots!
```

```
victim-web | 172.19.0.7 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
bot-2 | đang tấn công http://victim-web...

bot-1 | đang tấn công http://victim-web...
bot-4 | đang tấn công http://victim-web...
bot-3 | đang tấn công http://victim-web...

View in Docker Desktop View Config Enable Watch Detach

bot-1 | đang tấn công http://victim-web...
victim-web | 172.19.0.7 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
bot-1 | đang tấn công http://victim-web...
bot-3 | đang tấn công http://victim-web...
victim-web | 172.19.0.4 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
bot-2 | đang tấn công http://victim-web...

bot-3 | đang tấn công http://victim-web...
victim-web | 172.19.0.7 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
bot-1 | đang tấn công http://victim-web...
bot-2 | đang tấn công http://victim-web...
bot-3 | đang tấn công http://victim-web...

victim-web | 172.19.0.5 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"

bot-2 | đang tấn công http://victim-web...
victim-web | 172.19.0.7 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.4 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.6 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.6 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.4 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.4 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.6 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.4 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.4 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
victim-web | 172.19.0.4 - - [05/Apr/2026:11:24:58 +0000] "GET / HTTP/1.1" 200 896 "-" "python-requests/2.32.5" "-"
```

4.6.1 Trạng thái hệ thống trước tấn công

- *Mô tả:* Máy chủ **victim-web** hoạt động bình thường, không có truy cập lạ.
- *Thông số kỹ thuật:* * CPU Usage: Thấp (gần 0%).
 - Network I/O: Ổn định, chỉ phát sinh khi có người dùng hợp lệ truy cập.
 - Phản hồi người dùng: Truy cập **localhost:9090** tức thời (Latency < 100ms).

4.6.2 Diễn biến quá trình tấn công (Attack Execution)

Khi thực hiện lệnh `attack http://victim-web` từ `cc-server`, hệ thống ghi nhận các hiện tượng sau:

1. Sự bùng nổ của Log hệ thống:

- Log của `victim-web` ghi nhận hàng trăm yêu cầu `GET` / mỗi giây từ dải IP nội bộ của Docker (`172.19.0.x`).
- Tất cả các Bot đều sử dụng chung một định danh `User-Agent: python-requests/2.32.5`, cho thấy đây là truy cập không tự nhiên.

2. Chiếm dụng tài nguyên (Resource Exhaustion):

- Do máy chủ bị giới hạn `cpus: '0.1'`, ngay khi 20 Bot đồng loạt gửi yêu cầu, chỉ số CPU % trong `docker stats` vọt lên mức kịch trần (~10% của tổng tài nguyên máy hoặc 100% của mức giới hạn cho phép).
- RAM: Có xu hướng tăng nhẹ do Nginx phải duy trì hàng đợi (Connection Queue) cho các yêu cầu đang chờ xử lý.

4.6.3 Phân tích tình trạng (Degradation of Service)

Đây là điểm quan trọng nhất trong phân tích:

- **Hiện tượng:** Trình duyệt tại `localhost:9090` quay vòng liên tục (Loading) nhưng chưa báo lỗi ngay lập tức.
- **Nguyên nhân kỹ thuật:** * `Nginx Worker Processes`: Do Nginx là một web server hướng sự kiện (event-driven), nó vẫn cố gắng giữ các kết nối trong hàng đợi. Tuy nhiên, vì CPU bị giới hạn quá thấp (0.1), tốc độ xử lý mỗi request cực chậm.
 - **Trạng thái Treo (Stalling):** Yêu cầu của người dùng hợp lệ bị xếp hàng sau hàng ngàn yêu cầu của Botnet. Thời gian phản hồi (Response Time) tăng từ vài miligiây lên hàng chục giây, tạo ra trải nghiệm "tương đương với sập" đối với người dùng thực tế.

[illegible]

- *Giới hạn sập: Nếu tiếp tục duy trì hoặc tăng số lượng Bot (lên 50-100), hàng đợi của Nginx sẽ đầy (Listen Backlog full), lúc đó hệ thống mới chính thức trả về lỗi 502 Bad Gateway hoặc ngắt kết nối hoàn toàn.*

502 Bad Gateway

nginx

4.6.4 Kết luận kết quả Demo

- **Hiệu quả của đa luồng:** Dù chỉ với 5 Bot, nhưng nhờ kỹ thuật **threading**, số lượng yêu cầu tạo ra là cực lớn, đủ để làm nghẽn một Web Server cấu hình mặc định.
- **Tính nguy hiểm của Botnet:** Kẻ tấn công không cần máy tính cấu hình mạnh, chỉ cần số lượng lớn các thiết bị (Bot) cấu hình yếu cũng có thể đánh sập các hệ thống lớn.
- **Sự cần thiết của phòng thủ:** Thực nghiệm cho thấy nếu không có các lớp bảo mật như Firewall hay Rate Limiting, máy chủ hoàn toàn bất lực trước các cuộc tấn công tăng ứng dụng đơn giản như HTTP Flood.

Tiêu chí so sánh	Trạng thái bình thường (Normal)	Trạng thái bị tấn công (Under Attack)
Số lượng Request/giây	Rất thấp (chỉ từ người dùng thực)	Rất cao (Hàng trăm request từ 20+ Bots)
Tình trạng Log Nginx	Trống hoặc chỉ hiện vài dòng truy cập	Log chạy liên tục , lặp lại từ dải IP Bot
Sử dụng CPU (0.1 Limit)	< 1%	~10.04% (Chạm ngưỡng giới hạn)
Độ trễ (Latency)	< 100ms (Truy cập tức thì)	Rất cao (Quay vòng Loading lâu)
Trạng thái người dùng	Truy cập bình thường, ổn định	Bị nghẽn , không thể tải được nội dung
Mã phản hồi (HTTP Code)	200 OK	200 OK (nhưng xử lý cực chậm) hoặc 502/504

Trước khi bị tấn công:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
eba9f88d4542	victim-web	0.00%	5.945MiB / 50MiB	11.89%	2.66kB / 126B	7.82MB / 8.19kB	5

Sau khi bị tấn công (giới hạn cpu còn 1%):

```
services:
  victim-web:
    image: nginx:alpine
    deploy:
      resources:
        limits:
          cpus: '0.01'
          memory: 50M
    container_name: victim-web
    networks:
      - ddos_network
    ports:
      - "9090:80"
```

Command Prompt - docker stats victim-web							
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
eba9f88d4542	victim-web	1.48%	6.344MiB / 50MiB	12.69%	683kB / 1.66MB	7.82MB / 12.3kB	5

5. Đánh giá & kết luận

5.1 Kết quả đạt được

Dựa trên quá trình thực nghiệm, các mục tiêu đề ra ban đầu đã được hoàn thành:

- Xây dựng thành công mạng Botnet mô phỏng: Thiết lập được hệ thống phân tán gồm C&C Server, Victim và dàn Bot (từ 20 đến 100 bot) hoạt động đồng bộ trên nền tảng Docker.
- Chứng minh lỗ hổng tài nguyên: Xác nhận rằng khi giới hạn CPU ở mức thấp (0.1\$ CPU), máy chủ Web cực kỳ dễ bị tổn thương. Chỉ với lưu lượng truy cập từ 20 Bot, dịch vụ đã rơi vào trạng thái nghẽn (High Latency).

- Thu thập dữ liệu thực tế: Trích xuất được các chỉ số về CPU, RAM thông qua `docker stats` và hành vi tấn công thông qua Log Nginx, phục vụ tốt cho việc phân tích và nhận diện tấn công HTTP Flood.

5.2 Hạn chế mô phỏng

Mặc dù demo thành công, nhưng mô hình Lab này vẫn tồn tại một số hạn chế so với các cuộc tấn công thực tế (Real-world Attacks):

- Môi trường mạng nội bộ (Layer 2): Các Bot và Victim nằm chung trong một Docker Network, do đó không mô phỏng được các yếu tố như độ trễ internet (Network Latency), sự kiểm soát của các Router/Firewall của nhà cung cấp dịch vụ (ISP).
- Địa chỉ IP giả lập: Các IP của Bot (`172.19.0.x`) là IP nội bộ. Trong thực tế, kẻ tấn công thường sử dụng IP từ nhiều quốc gia khác nhau hoặc kỹ thuật IP Spoofing để vượt qua các bộ lọc IP đơn giản.
- Kịch bản tấn công đơn điệu: Hiện tại Bot chỉ thực hiện `GET` / liên tục. Các cuộc tấn công thực tế thường tinh vi hơn (như lướt trang, điền form, hoặc tấn công vào các API nặng) để đánh lừa các hệ thống phát hiện hành vi (WAF).
- Cấu hình máy chủ: Nginx trong Lab là cấu hình mặc định, chưa được tối ưu hóa hoặc cài đặt các module chống DDoS (như `limit_req` hay `limit_conn`), dẫn đến việc "sập" nhanh hơn so với thực tế.

5.3 Kết luận

Cuộc mô phỏng đã cung cấp một cái nhìn trực quan về mức độ nguy hiểm của tấn công DDoS tầng ứng dụng (Layer 7).

Những bài học quan trọng:

1. Tài nguyên là hữu hạn: Dù phần mềm có tối ưu đến đâu, nếu tài nguyên phần cứng (CPU/RAM) bị chiếm dụng hết bởi các yêu cầu rác, dịch vụ vẫn sẽ bị gián đoạn.
2. Tầm quan trọng của giám sát: Việc theo dõi Log và chỉ số tài nguyên theo thời gian thực là chìa khóa để phát hiện sớm các dấu hiệu bất thường trước khi hệ thống sụp đổ hoàn toàn.
3. Hướng phát triển: Để nâng cao tính bảo mật, các hệ thống cần triển khai các giải pháp phòng thủ đa tầng như:
 - Sử dụng Load Balancer để phân tán lưu lượng.

- Cấu hình giới hạn tốc độ truy cập (Rate Limiting) ngay tại tầng Web Server.
- Áp dụng các giải pháp lọc lưu lượng thông minh (WAF/Cloudflare) để loại bỏ Botnet từ xa.

Kết Luận Chung

Qua quá trình nghiên cứu và thực hiện đề tài “Tấn công Từ chối Dịch vụ Phân tán (DDoS) với Botnet mô phỏng”, có thể thấy rằng DDoS là một trong những mối đe dọa nghiêm trọng và phổ biến nhất trong lĩnh vực an toàn thông tin hiện nay. Với sự hỗ trợ của botnet, kẻ tấn công có thể huy động một số lượng lớn các máy bị nhiễm (zombie) để tạo ra lưu lượng truy cập khổng lồ, gây quá tải và làm gián đoạn hoạt động của hệ thống mục tiêu.

Thông qua việc phân tích kiến trúc botnet (bao gồm C&C Server và các bot), đề tài đã làm rõ cơ chế tổ chức, điều khiển và triển khai một cuộc tấn công DDoS. Bên cạnh đó, việc mô phỏng một hệ thống botnet đơn giản trong môi trường lab đã giúp minh họa trực quan cách thức các bot phối hợp gửi yêu cầu đến máy chủ, từ đó cho thấy mức độ ảnh hưởng rõ rệt đến hiệu năng hệ thống.

Tuy mô hình mô phỏng còn đơn giản và quy mô nhỏ, nhưng kết quả đạt được đã phản ánh phần nào tính nguy hiểm và khả năng mở rộng của các cuộc tấn công DDoS trong thực tế. Đồng thời, đề tài cũng đã đề xuất một số giải pháp phòng ngừa và giảm thiểu hiệu quả như sử dụng firewall, CDN, rate limiting và các hệ thống phát hiện xâm nhập.

Từ những kết quả trên, có thể khẳng định rằng việc hiểu rõ cơ chế hoạt động của botnet và DDoS không chỉ giúp nâng cao nhận thức về an ninh mạng mà còn là nền tảng quan trọng để xây dựng các hệ thống phòng thủ hiệu quả trong môi trường số ngày càng phức tạp hiện nay.

<i>Thành viên</i>	<i>Mã Sinh Viên</i>
<i>Nguyễn Nam Khánh</i>	<i>B24DCCCC160</i>
<i>Trương Duy Mạnh</i>	<i>B24DCCCC195</i>